

Curso de Especialización Ciberseguridad en las TIC

Tipos de vulnerabilidades en Software

Objetivos

- Detectar los problemas de los códigos.
- Identificar los riesgos.
- Buscar una solución a las vulnerabilidades.
- Probar que funciona correctamente sin vulnerabilidades.

Resumen

Autora: Marina del Pilar López Oliva

Noviembre 2025

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	11/11/2025	Realización del documento.

Índice de Contenidos

Ejercicio 1.....	4
Ejercicio 2.....	5
Ejercicio 3 - Gestión insegura de archivos.....	6
Ejercicio 4 - Autenticación insegura.....	7
Ejercicio 5 - Validación insuficiente.....	8
Ejercicio 6 - CSRF.....	9
Ejercicio 7 - Robo de datos sensibles.....	10
Ejercicio 8 - Manejo incorrecto de errores.....	11
Ejercicio 9 - Fuga de información por logs.....	12
Ejercicio 10 - Path Traversal.....	13

Ejercicio 1

El problema está en que el código es vulnerable a la inyección de SQL. Por lo tanto el atacante puede acceder a datos protegidos o modificar la base de datos. Habría que validar y comprobar que el usuario no pueda ejecutar código SQL. Para ello, deberemos utilizar los parámetros %s o ?. Esto elimina la inyección.

Solución del código:

```
import sqlite3

conexion = sqlite3.connect(":memory:")
cursor = conexion.cursor()
cursor.execute("CREATE TABLE empleados (id INTEGER, nombre TEXT, salario
INTEGER)")
cursor.execute("INSERT INTO empleados VALUES (?, ?, ?)", (1, 'Ana', 2000))
cursor.execute("INSERT INTO empleados VALUES (?, ?, ?)", (2, 'Luis', 2500))
conexion.commit()

campo = input("Campo (id, nombre, salario): ")
if campo in ("id", "nombre", "salario"):
    print("Campo no válido")
else:
    valor = input("Valor: ")
    consulta = f"SELECT * FROM empleados WHERE {campo} = ?"
    cursor.execute(consulta, (valor,))
    print(cursor.fetchall())

conexion.close()
```

Ejercicio 2

El problema en este ejercicio es el Cross-Site Scripting (XSS). El atacante puede ejecutar scripts peligrosos en el navegador de otros usuarios. Esto pasa cuando una app web muestra la entrada que ha introducido el usuario sin filtrar. Para solucionar esto debemos escapar los caracteres especiales. Con esto conseguimos que los scripts no se ejecuten, sino que se muestran como texto.

Solución del código:

```
import html

nombre = input("Nombre: ")
bio = input("Biografía: ")

nombre_filtrado = html.escape(nombre)
bio_filtrada = html.escape(bio)

html = f"<h1>{nombre_filtrado}</h1><p>{bio_filtrada}</p>"
print(html)
```

Ejercicio 3 - Gestión insegura de archivos

El problema en este caso es que permite la descarga sin validación de archivos. El código devuelve el archivo indicado sin validación, por lo que puede devolver cualquier archivo del sistema. Esto hace que sin importar si el usuario pueda o no acceder a él. Lo que puede hacer que el usuario acceda a información sensible del sistema.

Solución del código:

```
from flask import Flask, send_file, request
import os

app = Flask(__name__)

def leer_archivo_seguro(nombre):
    if not nombre.isalnum():
        return "Nombre no permitido."
    else:
        ruta = os.path.join("/usuario/archivos/", nombre + ".txt")

@app.route('/descarga')
def descarga():
    archivo = request.args.get(ruta)
    return send_file(archivo)
```

Ejercicio 4 - Autenticación insegura

Aquí el problema es que se realiza la registración del usuario y la contraseña en texto plano. Permite contraseñas poco robustas, sin cifrados por lo que es fácil averiguar la contraseña. Para ello lo que debemos hacer es permitir las contraseñas robustas y cifrarlas para almacenarlas.

Solución del código:

```
import hashlib

usuarios = {}

usuario = input("Usuarios a registrar: ")
contraseña = input("Contraseña: ")
contraseña_hash = hashlib.sha256(contraseña).hexdigest()
usuarios[usuario] = contraseña_hash

print(usuarios)
```

Ejercicio 5 - Validación insuficiente

La vulnerabilidad en este código es que simplemente comprueba si la edad es mayor, igual o menor a 18. Pero no valida si el usuario realmente ha escrito un número, por lo que se podría escribir texto o símbolos. El riesgo reside en que el programa puede fallar, romperse o incluso revelar información sensible. Para solucionarlo, vamos a validar que la entrada sea numérica y controlamos el error, así no pueden mandar datos erróneos ni romper la aplicación.

Solución del código:

```
edad = input("Edad: ")

if not isinstance(edad, (int, float))::
    return "Error: Solo se pueden validar números."
else:

    if int(edad) >= 18:
        print("Eres mayor de edad")
    else:
        print("Eres menor de edad")
```


Ejercicio 6 - CSRF

El problema está en que el programa cambia la contraseña sin comprobación ni confirmación. Cualquier sitio web puede lanzar esta función aunque no tenga permisos. Con esto el atacante puede engañar al usuario para que haga una opción sin querer, en este caso cambiar la contraseña, usando su sesión. Solucionar esto es hacer que la función sólo se ejecute si el usuario incluye el token CSRF correcto, evitando así peticiones no legítimas.

Solución del código:

```
def cambia_pwd(user, pwd):  
    print(f"Contraseña de {user} cambiada a {pwd}")  
  
usuario = input("Usuario: ")  
nueva = input("Nueva contraseña: ")  
token = input("Token de validación: ")  
  
def procesar_peticion(usuario, token):  
    token_sec = "token1234"  
  
    if token != token_sec:  
        print("Petición no permitida")  
    else:  
        cambia_pwd(usuario, nueva)
```

Ejercicio 7 - Robo de datos sensibles

La vulnerabilidad en este que se solicita y se envían credenciales por email sin cifrado. Esto permite que alguien los intercepte mientras viajan por la red. Con esto, el ciberdelincuente puede robar contraseñas o datos personales. Usaremos HTTPS y contraseñas cifradas para solucionar el problema.

Solución del código:

```
from smtplib import SMTP
import hashlib

def enviar_email(dest, mensaje):
    with SMTP("localhost") as smtp:
        smtp.sendmail("admin@ejemplo.com", dest, mensaje)

email = input("Destino: ")
usuario = input("Usuario: ")
pwd = input("Contraseña: ")

usu_cif = hashlib.sha256((usuario).encode()).hexdigest()
pwd_cif = hashlib.sha256((pwd).encode()).hexdigest()

texto = f"Usuario: {usu_cif} / Contraseña: {pwd_cif}"

enviar_email(email, texto)
```

Ejercicio 8 - Manejo incorrecto de errores

Este programa muestra detalles internos de errores, lo que puede usar un usuario para atacar. Se filtra información técnica que ayuda a atacar el sistema, por lo que un ciberdelincuente sólo tiene que hacerse con está información para atacar al programa. Solucionamos esta vulnerabilidad con un mensaje genérico para todos los errores, sin especificar los errores internos ni las técnicas del error.

Solución del código:

```
a = input("Cociente: ")
```

```
b = input("Divisor: ")
```

```
try:
```

```
    print(float(a) / float(b))
```

```
except Exception:
```

```
    print("Ha ocurrido un error inesperado. Por favor, contactar con soporte o  
    inténtelo más tarde")
```

Ejercicio 9 - Fuga de información por logs

La vulnerabilidad se encuentra en cómo se registra la contraseña, en texto plano. Lo mismo que hemos encontrado en ejercicios anteriores, pero en este caso ocurre en el log. Para esto, debemos cifrar la contraseña antes de registrarla.

Solución del código:

```
import logging

logging.basicConfig(level=logging.INFO)

usuario = input("Usuario:")
pwd = input("Contraseña:")

usu_cif = hashlib.sha256((usuario).encode()).hexdigest()
pwd_cif = hashlib.sha256((pwd).encode()).hexdigest()

logging.info(f"Login de usuario: {usu_cif} con contraseña: {pwd_cif}")

print("Login registrado")
```

Ejercicio 10 - Path Traversal

Aquí el problema reside en que permite al usuario poner rutas y borrar ficheros fuera de la carpeta. Lo que permite a los usuario poder borrar algún archivo con información sensible fuera de la carpeta del programa. Incluso puede borrar archivos críticos del sistema. Para solucionar la gestión insegura de archivos, vamos a hacer que no permita rutas ni caracteres peligrosos, solo archivos internos y controlados.

Solución del código:

```
import os

nombre = input("Archivo a eliminar: ")

def borrar_archivo_seguro(nombre):
    if not nombre.isalnum():
        return "Nombre no permitido."
    else:
        os.remove("archivos/" + nombre)
        print("Borrado")
```